Lambdas, Pure vs Impure Functions

Lambdas are tiny anonymous functions; pure functions depend only on their inputs and change nothing else.

Lambda syntax: lambda params: expression

```
# define + call at once
print((lambda x: x ** 2)(5))
# defaults are allowed
print((lambda a, b=10: a + b)(1))
print((lambda *xs: sum(xs))(1, 2, 3))  # so is *args
```

```
OUTPUT
25
11
6
```

	`def`	`lambda`
Body	many statements	**one expression** (value is returned)
Name	its own (`__name__`)	`<lambda>`
Docstring / hints	yes	no
Best for	reusable logic	short throw-away callbacks

Where lambdas shine: key= and helpers

```
people = [("Ann", 31), ("Bob", 25), ("Cy", 28)]
print(sorted(people, key=lambda p: p[1]))
print(max(people, key=lambda p: p[1]))

nums = [1, 2, 3, 4, 5, 6]
print(list(map(lambda n: n * n, nums)))
print(list(filter(lambda n: n % 2 == 0, nums)))
```

```
OUTPUT
[('Bob', 25), ('Cy', 28), ('Ann', 31)]
('Ann', 31)
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
```

```
from functools import reduce
print(reduce(lambda acc, n: acc * n, [1, 2, 3, 4]))
```

```
OUTPUT
24
```

TIP A comprehension is usually clearer than `map`/`filter` + lambda: `[n * n for n in nums]`. PEP 8: don't assign a lambda to a name – use `def`.

PITFALL Late binding: a lambda looks up its variables when **called**, not when created.

```
fs = [lambda: i for i in range(3)]
# all see the final i
print([f() for f in fs])
fs = [lambda i=i: i for i in range(3)]
# default binds i now
print([f() for f in fs])
```

```
OUTPUT
[2, 2, 2]
[0, 1, 2]
```

Pure vs impure

	Pure	Impure
Output depends on	only its arguments	arguments **+ hidden state** (globals, time, random, files)
Side effects	none	mutates arguments/globals, prints, I/O, network
Same input → same output	always	not guaranteed
Testing / caching	easy and safe	harder; unsafe to cache

```
def add_pure(items, x):
    # new list; input untouched
    return items + [x]

tasks = ["a"]
def add_impure(x):
    # changes state outside itself
    tasks.append(x)

print(add_pure(tasks, "b"), tasks)
add_impure("c")
print(tasks)
```

```
OUTPUT
['a', 'b'] ['a']
['a', 'c']
```

```
nums = [3, 1, 2]
# pure: returns a new list
print(sorted(nums), nums)
# impure: in place, returns None
nums.sort()
print(nums)
```

```
OUTPUT
[1, 2, 3] [3, 1, 2]
[1, 2, 3]
```

REMEMBER Keep the core logic pure; push impurity (I/O, randomness, time) to the edges of your program.